Functional Programming

Lars-Henrik Eriksson*

2021-10-15

Introduction

We will define a natural semantics for a pure functional language **Func** with first-class functions and static scope rules, loosely modeled on Haskell – but without lazy evaluation. The language will have two basic kinds of data, integers and boolean, as well as pairs of objects (including "cons" cells for lists) and functions. Initially, the language will be untyped – we will add types later.

Syntactic domains

We will have three meta-variables with the corresponding syntactic categories:

n will range over numerals **Num**,

x will range over identifiers **Id**,

E will range over expressions **Exp**

As with the imperative languages, meta-variables can be primed or subscripted when we need more than one.

Num is the same as that used with the imperative languages. **Id** is the same as **Var**, but it is called **Id** instead, as pure functional languages don't have variables. Note that we will assume an extended concrete syntax for **Id** compared to **Var** – see below.

*Generously translated into LaTeX by Kristoffer Jansson in 2024

The abstract syntax for expressions is:

$E ::= n \mid x \mid \textbf{if } E_0 \textbf{ then } E_1 \textbf{ else } E_2 \mid (E_1, E_2) \mid E_1\ E_2 \mid \lambda x \to E \mid \textbf{rec } x \to E$

The forms n and x are straightforward. The if-form is the standard conditional expression. (E_1, E_2) means a pair of the two expressions E_1 and E_2. The pair form lets us construct composite data structures such as lists.

$E_1\ E_2$ is function application (call) where E_1 is an expression evaluating to the function to be called and E_2 is an expression evaluating to the argument of the function. Following Haskell, the function and argument are simply written next to each other. If you want to use the more common syntax of enclosing the argument in parenthesis, that's fine. (The parenthesis will not have any special meaning – they'll be seen just as any other parenthesis used for grouping.)

If you compare with the abstract syntax of arithmetic and boolean expressions of the imperative languages, you'll see that the syntax doesn't name any particular functions or operators such as +, –, ==, etc. Syntactically, these operators will be identifiers like any other. Of course the concrete syntax must admit identifiers with "symbolic" names such as +. (The concrete syntax isn't our concern – we'll just assume that there exists a suitable one.)

One important difference from Haskell is that the symbol **::** is used as the list constructor operator and not as the typing operator. The reason is that the most common symbol for typing is **:** so we want to reserve that operator for that purpose.

Again following Haskell, all functions have exactly one argument. If a function needs more than one piece of data, we have two possibilities. Either we use currying which is the norm in Haskell, or we can use the pair form to "package" the data into a single argument. In contrast to Haskell, **Func** will use the "packaged" form for built-in functions as this simplifies the semantics.

For readability, when writing expressions we will use infix operators in the usual way, e.g. we will write $x - 1$ or x<=0 which should be understood to actually mean $(-)(x, 1)$ and (<=)$(x, 0)$.

$\lambda x \to E$ is functional abstraction – or an "anonymous function". It is an expression which is a function of one argument x and body E. When called, E is evaluated with x being bound to the argument. For simplicity, **Func** will not have pattern matching or multiple clauses. The language also does not

provide any way to specifically define named functions, but we'll see shortly how we can obtain named functions anyway.

rec $x \to E$ is a recursion operator. E should be a function abstraction. The value of rec $x \to E$ is the same function abstraction, but with x bound to E itself so that the function can call itself recursively using the name x. The use of **rec** is described below.

Example F.1

$(\lambda x \to x * x)\ (1 + 1)$ is an expression that evaluates to 4. The expression consists of a function call. The function is $(\lambda x \to x * x)$ and the argument is the value of $1 + 1$ which is 2. The called, $x * x$, is evaluated with x bound to 2, so the result will be $2 * 2$ or 4. □

Definitions

The language **Func** will not have global definitions of functions or other values – all function must be defined locally in the expression where they are used. E.g. if we want to compute the factorial of 4, we could write

```
let
  fact x = if x == 0 then 1 else x*fact(x-1)
in
  fact 4
```

But since there are neither *let* expressions nor definitions in the abstract syntax for **Func**, how is this done?

It is common in semantic descriptions to simplify the actual semantics by defining a "core" language and rewriting other language forms to forms of the core. The advantage is that the semantic description can be made smaller and any proofs about the semantics are smaller. The disadvantage is that you can generally not apply the semantics directly to a program – it must first be rewritten to only use the core language features.

Definitions of functions with arguments can be rewritten as definitions of a value which is an anonymous functions. I.e. a definition

$x\ x' = E$

can be rewritten to the following equivalent definition

$x = \mathbf{rec}\ x \to \lambda x' \to E$

The **rec** operator makes it possible for the function to call itself recursively using the name x. (See the semantic rule for **rec** below.) Finally x is bound to the function. The double occurrence of x is because the scopes of the two occurrences are different.

Next, the let-expressions can be rewritten using anonymous functions and function application. The let-expression

```
let
  x = E
in
  E'
```

can be rewritten to the following expression, which will compute the same value:

$(\lambda x \to E')\ E$

Example F.2

```
let
  a = 4
in
  a + 1
```

The intended meaning of this expression is that **a** is bound to 4 in the expression $a+1$, so the computed value will be 5. Using the rewrite rule above, we get $(\lambda a \to a+1)\ 4$, which indeed evaluates to 5.

Example F.3

```
let
  fact x = if x == 0 then 1 else x*fact(x-1)
in
  fact 4
```

This is rewritten in two steps. First to

```
let
  fact =
    rec fact →
      λx → if x == 0 then 1 else x*fact(x-1)
in
  fact 4
```

then to

```
(λfact → fact 4)
(rec fact → λx → if x == 0 then 1 else x*fact(x-1))
```

which does compute the factorial of 4. □

Example F.4

An expression for the standard append function on lists can be written as:

```
rec append → λx → λy →
  if x == [] then
    y
  else
    hd x :: append (tl x) y
```

□

Semantic domains

In the procedural languages we've seen that the only kind of data was integers. *if* and *while* statements did have boolean expressions but boolean values could not be assigned to variables. In **Func**, we will have several kinds of data. Firstly, we don't distinguish between arithmetic and boolean expressions, so any expression could have a value of either type. Secondly, there are the pairs and function values. Thus in **Func**, the value of an expression will belong to a more complex set **Val**, which is built up from various other sets. The meta-variable v will range over **Val**.

$$\mathbf{Val} = \boldsymbol{Z} \cup \boldsymbol{T} \cup \mathbf{Pair} \cup \{\mathbf{nil}\} \cup \mathbf{Fun} \cup \mathbf{RecFun} \cup \mathbf{Builtin}$$

$$\mathbf{Pair} = \mathbf{Val} \times \mathbf{Val}$$

Z and **T** are the same sets of integers and booleans that we have seen before. Pair is simply the set of pairs of two other values and nil is a special object used to mark the end of lists (see below). We'll return to **Fun**, **RecFun** and **Builtin**.

Recall that in the semantics for the imperative languages **Block** and **Proc**, we used environments to keep track of name bindings while states (or stores) were used to keep track of the values of variables. In a pure functional language such as **Func**, we don't have variables so we don't need a state (or store). We do need an environment since we have name bindings. An environment *env* will be an element of

$$\mathbf{Env} = \mathbf{Id} \hookrightarrow \mathbf{Val}$$

Thus the environment will look very much like a combination of the state and environment used in the language **Proc** with dynamic scope rules. But at the very beginning I said that **Func** had static scope rules. The language **Proc** with static scope rules saved the environment in which a procedure was defined to ensure that the correct bindings were used when it was called. Do we not need the same here?

In fact we do, but it is handled slightly differently. In **Func**, there are no function definitions as such – function abstractions are just expressions. Whenever a function abstraction is encountered during the execution of a **Func** program, a value will be computed which includes all the necessary information to correctly call the function. These values belong to the set **Fun**.

$$\mathbf{Fun} = \mathbf{Id} \times \mathbf{Exp} \times \mathbf{Env}$$

Thus a "function value" is a triplet including

- The name of the argument of the function. (We didn't need this in **Proc**, as procedures in **Proc** do not have arguments.)
- The expression which is the body of the function. (Compare this to the statement in **Proc** procedure environments.)
- The environment at the time time the function value was computed. (Compare this to the variable and procedure environments saved together with the procedure definition in **Proc** procedure environments.)

So if an identifier X in **Fun** is bound to a function in environment env, then $env(x) = (x', E, env')$. Compare this to **Proc**, e.g. the side conditions of table 3.4. Further on, we'll see how this works in detail.

Recursive functions work similarly but belong to the set **RecFun** of quadruplets instead.

$$\mathbf{RecFun} = \mathbf{Id} \times \mathbf{Exp} \times \mathbf{Id} \times \mathbf{Env}$$

The difference is the additional (second) identifier which is the name which the recursive function can use to call itself.

Finally, we shall address the built-in functions. In the imperative languages, the built-in operators (+, –, = etc) had their own syntax. In **Func**, there is

no special syntax for them. Instead we assume that there is a initial binding environment env_0, where all built-in functions are defined. Their names are bound to special values which are used in the semantic rules to determine what built-in function should be evaluated. These special values are in the set **Builtin**. Their use will become clear when we define the semantic rules. The meta-variable b will range over **Builtin**.

$$\mathbf{Builtin} = \{\text{plus, minus, times, divide, equals, lesseq, fst, snd}\}$$

The intended meaning of all except the last two should be obvious from their names. *fst* and *snd* represent functions that return the first or second component of a pair, respectively.

Semantic rules

The semantic rules and axiom schemas define a transition relation from an expression E to a value v in a binding environment env. It has the form

$$env \vdash E \longrightarrow v$$

and the rules and schemas are as follows:

[num] $env \vdash n \longrightarrow \mathcal{N}[\![n]\!]$

[id] $env \vdash x \longrightarrow env(x)$ if env is defined on x.

[if$^{\mathbf{tt}}$] $$\frac{env \vdash E \longrightarrow \mathrm{tt} \qquad env \vdash E_1 \longrightarrow v}{env \vdash \text{if } E \text{ then } E_1 \text{ else } E_2 \longrightarrow v}$$

[if$^{\mathbf{ff}}$] $$\frac{env \vdash E \longrightarrow \mathrm{ff} \qquad env \vdash E_2 \longrightarrow v}{env \vdash \text{if } E \text{ then } E_1 \text{ else } E_2 \longrightarrow v}$$

[pair] $$\frac{env \vdash E_1 \longrightarrow v_1 \qquad env \vdash E_2 \longrightarrow v_2}{env \vdash (E_1, E_2) \longrightarrow (v_1, v_2)}$$

[fun] $env \vdash \lambda x \longrightarrow E \longrightarrow (x, E, env)$

[rec] $$\frac{env \vdash E \longrightarrow (x', E', env')}{env \vdash rec\ x \rightarrow E \longrightarrow (x', E', x, env')}$$

[app] $$\frac{env \vdash E_1 \longrightarrow (x, E', env') \qquad env \vdash E_2 \longrightarrow v' \qquad env'[x \mapsto v'] \vdash E' \longrightarrow v}{env \vdash E_1 E_2 \longrightarrow v}$$

[app$^{\mathbf{rec}}$] $$\frac{\begin{array}{c} env \vdash E_1 \longrightarrow (x, E', x', env') \\ env \vdash E_2 \longrightarrow v' \\ env'[x' \mapsto (x, E', x', env')][x \mapsto v'] \vdash E' \longrightarrow v \end{array}}{env \vdash E_1 E_2 \longrightarrow v}$$

[app$^{\mathbf{bi}}$] $$\frac{env \vdash E_1 \longrightarrow b \qquad env \vdash E_2 \longrightarrow v'}{env \vdash E_1 E_2 \longrightarrow v} \text{ where } v = \text{ builtin}(b, v')$$

The semantic function builtin is defined like this:

$$
\begin{aligned}
\text{builtin}(b, v) &= v_1 + v_2 && \text{if b = plus and } v = (v_1, v_2) \text{ and } v_1, v_2 \in \mathbb{Z} \\
&= v_1 - v_2 && \text{if b = minus and } v = (v_1, v_2) \text{ and } v_1, v_2 \in \mathbb{Z} \\
&= v_1 \cdot v_2 && \text{if b = times and } v = (v_1, v_2) \text{ and } v_1, v_2 \in \mathbb{Z} \\
&= v_1 / v_2 && \text{if b = divide and } v = (v_1, v_2) \text{ and } v_1, v_2 \in \mathbb{Z} \text{ and } v_2 \neq 0 \\
&= \text{tt} && \text{if b = equals and } v = (v_1, v_2) \text{ and } v_1 = v_2 \\
&= \text{ff} && \text{if b = equals and } v = (v_1, v_2) \text{ and } v_1 \neq v_2 \\
&= \text{tt} && \text{if b = lesseq and } v = (v_1, v_2) \text{ and } v_1, v_2 \in \mathbb{Z} \text{ and } v_1 \leq v_2 \\
&= \text{ff} && \text{if b = lesseq and } v = (v_1, v_2) \text{ and } v_1, v_2 \in \mathbb{Z} \text{ and } v_1 > v_2 \\
&= v_1 && \text{if b = fst and } v = (v_1, v_2) \\
&= v_2 && \text{if b = snd and } v = (v_1, v_2)
\end{aligned}
$$

The rules and axiom schemas [num], [id], [if^{tt}], [if^{ff}] and [pair] are straightforward. The remaining rules and axiom schemas will need some explanation.

[fun]: The "value" of a function abstraction is simply the function abstraction itself in a different "packaging". Instead of the abstract syntax expression $\lambda x \to E$ we get a triplet with x and E and also the current binding environment. This is needed for static scoping so that we can restore that binding environment when the function is called.

[rec]: The recursion operator turns a function value triplet into a quadruplet which additionally includes the identifier which can be used within the function to recursively call itself. The actual mechanism to achieve this is provided by the [app^{rec}] rule.

[app]: This is the "standard" rule for function applications (calls). Both the function expression and its arguments are evaluated. Then the body of the function is evaluated in its definition environment but with its argument identifier being bound to the argument of the call.

[app^{rec}]: A variation of the [app] rule for recursive functions. The function body is evaluated in an environment where an identifier is bound to the function itself so that it can call itself recursively.

[app^{bi}]: A variation of the [app] rule for built-in functions. If the function expression E_1 evaluates to one of the special values in the set Builtin associated with built-in functions, then the semantic function builtin is used to compute the built-in function.

The initial binding environment env_0 mentioned above will be:

$$\begin{aligned}\text{env}_0 = \Big[\, [\,] \mapsto \texttt{nil}, \texttt{true} \mapsto \texttt{tt}, \texttt{false} \mapsto \texttt{ff}, \texttt{==} \mapsto \texttt{equals}, \texttt{<=} \mapsto \texttt{lesseq},\\ + \mapsto \texttt{plus}, - \mapsto \texttt{minus}, * \mapsto \texttt{times}, / \mapsto \texttt{divide},\\ \texttt{fst} \mapsto \mathit{fst}\,, \texttt{snd} \mapsto \mathit{snd}\,, \texttt{hd} \mapsto \mathit{fst}\,, \texttt{tl} \mapsto \mathit{snd}\,, :: \mapsto (x, x, \emptyset)\Big]\end{aligned}$$

Note that the list functions hd and tl are other names for the pair access functions fst and snd. This is because pairs can be used to build lists. (Remember that we are so far dealing with an untyped language!)

Also, the list construction operator **(::)** is defined as the identity function – meaning that it will just return the pair of arguments it is called with. The reason for this will become clear when we later introduce types.

Example F.5

Show that $\text{env}_0 \vdash 1 + 2 \longrightarrow 3$

$$\dfrac{\overset{[\text{id}]}{\text{env}_0 \vdash + \longrightarrow \mathit{plus}} \qquad \dfrac{\overset{[\text{num}]}{\text{env}_0 \vdash 1 \longrightarrow 1} \qquad \overset{[\text{num}]}{\text{env}_0 \vdash 2 \longrightarrow 2}}{\text{env}_0 \vdash (1,2) \longrightarrow (1,2)}\ \text{pair}}{\text{env}_0 \vdash 1 + 2 \longrightarrow 3}\ [\text{app}^{bi}]\ \text{where } 3 = \text{builtin(plus, (1,2))}$$

Note that $\text{env}_0 \vdash x \longrightarrow \text{plus}$ is a valid instance of the [id] axiom schema as $\text{env}_0(+) = \text{plus}$. □

Example F.6

Show that $\text{env}_0 \vdash (\lambda a \to a + 1)\ 4 \longrightarrow 5$.

$$\dfrac{\overset{[\text{fun}]}{\text{env}_0 \vdash \lambda a \to a + 1 \longrightarrow (a, a + 1, \text{env}_0)} \qquad \overset{[\text{num}]}{\text{env}_0 \vdash 4 \longrightarrow 4} \qquad \dfrac{\overset{[\text{id}]}{\text{env}_0[a \mapsto 4] \vdash + \longrightarrow \mathit{plus}} \qquad \dfrac{\overset{[\text{id}]}{\text{env}_0[a \mapsto 4] \vdash a \longrightarrow 4} \qquad \overset{[\text{num}]}{\text{env}_0[a \mapsto 4] \vdash 1 \longrightarrow 1}}{\text{env}_0[a \mapsto 4] \vdash (a, 1) \longrightarrow (4, 1)}}{\text{env}_0)[a \mapsto 4] \vdash a + 1 \longrightarrow 5}\ [\text{app}^{\text{bi}}]\ 5 = \text{builtin(plus, (4,1))}}{\text{env}_0 \vdash (\lambda a \to a + 1)\ 4 \longrightarrow 5}\ [\text{app}]$$

Note that $\text{env}_0[a \mapsto 4] \vdash a \longrightarrow 4$ is a valid instance of the [id] axiom schema as $\text{env}_0[a \mapsto 4](a) = 4$.

Example F.7

Show that $env_0 \vdash (\lambda fact \to fact\ 1)\ (\mathbf{rec}\ fact \to \lambda x \to \mathbf{if}\ x{=}{=}0\ \mathbf{then}\ 1\ \mathbf{else}\ x * fact\ (x-1)) \longrightarrow 1$. (A boring example, but enough to show a recursive call.) We use the following shorthands:

$$f = \mathbf{if}\ x{=}{=}0\ \mathbf{then}\ 1\ \mathbf{else}\ x * fact\ (x-1)$$

$$env_1 = env_0[fact \mapsto (x, f, fact, env_0)][x \mapsto 1]$$

$$env_2 = env_0[fact \mapsto (x, f, fact, env_0)][x \mapsto 0]$$

To save space, side conditions of [app^{bi}] are not written out in this derivation. Names of axioms are also omitted.

$$[\text{app}]\ \frac{env_0 \vdash \lambda fact \to fact\ 1 \longrightarrow (fact, fact\ 1, env_0) \qquad [\text{rec}]\ \dfrac{env_0 \vdash \lambda x \to f \longrightarrow (x, f, env_0)}{env_0 \vdash \mathbf{rec}\ fact \to \lambda x \to f \longrightarrow (x, f, fact, env_0)} \qquad T_1}{env_0 \vdash (\lambda fact \to fact\ 1)\ (\mathbf{rec}\ fact \to \lambda x \to f) \longrightarrow 1}$$

The placeholder tree T_1 shows the initial call of the factorial function:

$$[\text{app}^{rec}]\ \frac{env_0[fact \mapsto (x, f, fact, env_0)] \vdash fact \longrightarrow (x, f, fact, env_0) \qquad env_0[fact \mapsto (x, f, fact, env_0)] \vdash 1 \longrightarrow 1 \qquad T_2}{env_0[fact \mapsto (x, f, fact, env_0)] \vdash fact\ 1 \longrightarrow 1}$$

The placeholder tree T_2 shows the evaluation of the body in the first call of the factorial function:

$$[\text{if}^{ff}]\ \frac{[\text{app}^{bi}]\ \dfrac{env_1 \vdash x \longrightarrow 1 \qquad env_1 \vdash 0 \longrightarrow 0}{env_1 \vdash x{=}{=}0 \longrightarrow \texttt{false}} \qquad [\text{app}^{bi}]\ \dfrac{env_1 \vdash x \longrightarrow 1 \qquad T_3}{env_1 \vdash x * fact\ (x-1) \longrightarrow 1}}{env_1 \vdash \mathbf{if}\ x{=}{=}0\ \mathbf{then}\ 1\ \mathbf{else}\ x * fact\ (x-1) \longrightarrow 1}$$

The placeholder tree T_3 shows the recursive call of the factorial function:

$$[\text{app}^{rec}]\ \frac{env_1 \vdash fact \longrightarrow (x, f, fact, env_0) \qquad [\text{app}^{bi}]\ \dfrac{env_1 \vdash x \longrightarrow 1 \qquad env_1 \vdash 1 \longrightarrow 1}{env_1 \vdash x - 1 \longrightarrow 0} \qquad T_4}{env_1 \vdash fact\ (x-1) \longrightarrow 1}$$

Finally, the placeholder tree T_4 shows the termination of the factorial function:

$$[\text{if}^{tt}]\ \frac{[\text{app}^{bi}]\ \dfrac{env_2 \vdash x \longrightarrow 0 \qquad env_2 \vdash 0 \longrightarrow 0}{env_2 \vdash x{=}{=}0 \longrightarrow \texttt{true}} \qquad env_2 \vdash 1 \longrightarrow 1}{env_2 \vdash \mathbf{if}\ x{=}{=}0\ \mathbf{then}\ 1\ \mathbf{else}\ x * fact\ (x-1) \longrightarrow 1}$$

□

Question F.8

In **Func**, execution errors are possible. What happens if we try to determine the meaning of a program where an execution error would occur at runtime? What would happen if infinite recursion occurred? □

Question F.9

Compare using the semantics of **While** to determine the meaning an arithmetic or boolean expression compared to determine the meaning of the corresponding expression in **Func**. Does the case of **Func** seems much more complicated? What is the reason for this complication? □

Question F.10

Prove, using induction over the shape of derivation trees, that the semantics of **Func** is deterministic. Well, you don't actually have to do the proof, but spend a little while thinking about it. Would the proof be different in principle from the corresponding proof for **While** (Theorem 2.9)? □

Structured Operational Semantics

A natural semantics is, well, natural for a pure functional language since there is no notion of state change and thus no sequencing (at least not in the abstract sense). It is also possible to write a small-step semantics with individual execution steps. I will only outline the basic ideas here.

A configuration is a single **Func** expression – we use no environments. The final configurations are those expressions which are just values that can't be evaluated further. Thus each execution step involves applying a function to arguments, construction a pair or making a choice.

Applying a function abstraction (λ) to arguments involves *replacing* the entire function application with the body of the function and simultaneously *replacing* the bound variables with the arguments. The replacement is the reason we can do without environments. This is sometimes called a *rewriting semantics*.

Similarly the recursion operator (**rec**) is rewritten by replacing the name of the recursive function with its own definition – thus making recursion possible.

It is important that only the outermost (free) scope of identifiers are replaced. Thus the binding operators (λ, **rec**) "block" replacement of the identifiers that they bind.

I will not show the actual rules here – just a simple example. The expression to be rewritten in each step is underlined. Note how the rewriting is done.

Example F.12

Show that the program
$(\lambda fact \to fact\ 1)\ (\mathbf{rec}\ fact \to \lambda x \to \mathbf{if}\ x{=}{=}0\ \mathbf{then}\ 1\ \mathbf{else}\ x * fact(x-1))$
reduces to 1 using structured operational semantics:

$(\lambda fact \to fact\ 1)\ (\mathbf{rec}\ fact \to \lambda x \to \mathbf{if}\ x{=}{=}0\ \mathbf{then}\ 1\ \mathbf{else}\ x * fact(x-1)) \Rightarrow$

$(\mathbf{rec}\ fact \to \lambda x \to \mathbf{if}\ x{=}{=}0\ \mathbf{then}\ 1\ \mathbf{else}\ x * fact(x-1))\ 1 \Rightarrow$

$(\lambda x \to \mathbf{if}\ x{=}{=}0\ \mathbf{then}\ 1\ \mathbf{else}\ x *$

$\quad (\mathbf{rec}\ fact \to \lambda x \to \mathbf{if}\ x{=}{=}0\ \mathbf{then}\ 1\ \mathbf{else}\ x * fact(x-1))(x-1))\ 1 \Rightarrow$

$\mathbf{if}\ 1{=}{=}0\ \mathbf{then}\ 1\ \mathbf{else}\ 1*$

$\quad (\mathbf{rec}\ fact \to \lambda x \to \mathbf{if}\ x{=}{=}0\ \mathbf{then}\ 1\ \mathbf{else}\ x * fact(x-1))(1-1)) \Rightarrow$

$\mathbf{if}\ \texttt{false}\ \mathbf{then}\ 1\ \mathbf{else}\ 1*$

$\quad (\mathbf{rec}\ fact \to \lambda x \to \mathbf{if}\ x{=}{=}0\ \mathbf{then}\ 1\ \mathbf{else}\ x * fact(x-1))(1-1)) \Rightarrow$

$1 * (\mathbf{rec}\ fact \to \lambda x \to \mathbf{if}\ x{=}{=}0\ \mathbf{then}\ 1\ \mathbf{else}\ x * fact(x-1))(1-1) \Rightarrow$

$1 * \lambda x \to \mathbf{if}\ x{=}{=}0\ \mathbf{then}\ 1\ \mathbf{else}\ x*$

$\quad (\mathbf{rec}\ fact \to \lambda x \to \mathbf{if}\ x{=}{=}0\ \mathbf{then}\ 1\ \mathbf{else}\ x * fact(x-1))(x-1)(1-1) \Rightarrow$

$1 * \lambda x \to \mathbf{if}\ x{=}{=}0\ \mathbf{then}\ 1\ \mathbf{else}\ x*$

$\quad (\mathbf{rec}\ fact \to \lambda x \to \mathbf{if}\ x{=}{=}0\ \mathbf{then}\ 1\ \mathbf{else}\ x * fact(x-1))(x-1)(0) \Rightarrow$

$1 * \mathbf{if}\ 0{=}{=}0\ \mathbf{then}\ 1\ \mathbf{else}\ 0*$

$\quad (\mathbf{rec}\ fact \to \lambda x \to \mathbf{if}\ x{=}{=}0\ \mathbf{then}\ 1\ \mathbf{else}\ x * fact(x-1))(0-1) \Rightarrow$

$1 * \mathbf{if}\ \texttt{true}\ \mathbf{then}\ 1\ \mathbf{else}\ 0*$

$\quad (\mathbf{rec}\ fact \to \lambda x \to \mathbf{if}\ x{=}{=}0\ \mathbf{then}\ 1\ \mathbf{else}\ x * fact(x-1))(0-1) \Rightarrow$

$1 * 1 \Rightarrow$

1